



Höskolan Kristianstad
291 88 Kristianstad
044 20 30 00
www.hkr.se

Movie Tracker App

Full stack Project

Student: Mohammed Nour Cuneyt Omar

Teacher: Deema Aloom

Date: 20/04/2026

System Overview

Movie Tracker is a fullstack web application built with React, Express, and MongoDB. It allows users to manage movies by adding, editing, deleting, and viewing them. The system also supports user reviews, where each movie can have multiple reviews. Additionally, users can filter movies by genre, rating, and release year. The application is designed for movie enthusiasts who want a simple and organized way to track and evaluate films.

Problem & Solution

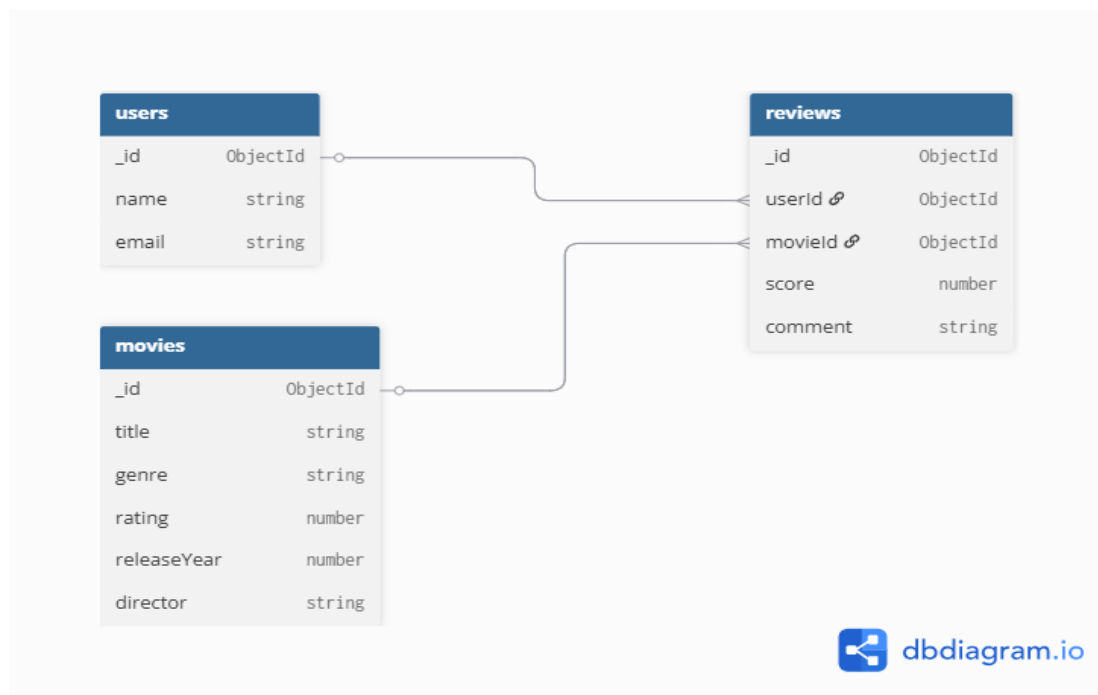
People who watch many movies often struggle to remember what they watched and how they rated them. Existing tools are either too complex or lack proper organization. This application solves the problem by providing a clean interface to manage movies and reviews in one place using a modern fullstack architecture (React frontend, Express backend, MongoDB database).

Database Design

The database consists of three collections: Users, Movies, and Reviews. Users store name and email. Movies store title, genre, release year, rating, and director, with validation rules such as required fields, enum for genre, and rating limits (0–10). Each movie is linked to a user using an ObjectId reference. Reviews connect users and movies using ObjectId references (userId and movieId) and include score and comment. A user can create many reviews, and a movie can have many reviews. Figure 1 shows the ERD diagram representing these relationships.

ERD Diagram

Figure 1: ERD Diagram of the Movie Tracker Application



This diagram shows the relationships between Users, Movies, and Reviews, where reviews link users and movies using ObjectId references.

API Endpoints

GET /api/movies: Description: Retrieves all movies with optional filters (genre, rating).

POST /api/movies: Description: Creates a new movie.

PUT /api/movies/:id: Description: Updates an existing movie.

Example API Request & Response: Create Movie: POST /api/movies

Request: {"title": "Inception", "genre": "Sci-Fi", "rating": 9, "releaseYear": 2010, "director": "Christopher Nolan"}

Response: {"_id": "...", "title": "Inception", "genre": "Sci-Fi", "rating": 9, "releaseYear": 2010, "director": "Christopher Nolan"}

Reflection

One of the main challenges was managing communication between frontend and backend, especially handling asynchronous data and UI updates. Initially, the UI lacked proper loading and error handling, which caused confusion. This was solved by implementing loading states and error messages in React. Another challenge was structuring the backend using Router → Controller → Model, which improved code organization and maintainability.

From this experience, I learned how to properly connect frontend and backend, handle asynchronous data using React, and structure backend code using the Router → Controller → Model pattern. I also improved my understanding of REST APIs and error handling.

Feature Iteration

Initially, the application returned all movies without filtering, making it difficult for users to find specific results. Later, filtering by genre and rating was implemented using query parameters in the API, significantly improving usability and allowing users to retrieve more relevant movies.

Relevant commits:

- d12fc33: added filtering feature
- 2f36490: improved rating filtering

Conclusion

This project demonstrates the development of a fullstack application using modern technologies. It highlights key concepts such as CRUD operations, REST API design, database relationships, and frontend-backend integration. The application solves a real problem and can be further extended with features like authentication and advanced analytics.